

1. Which of the following statements is correct for Java multi-threading programming:

- A. A thread is in „runnable thread” state, if and only if, the programmer calls the start() method from an object which is instance from a class that is derived from the Thread class
- B. For a thread to be in "runnable thread” state, it is sufficient to create an object from a class that is derived from the Thread class
- C. For a thread to be in "runnable thread” state, it is sufficient to create an object from a class that is implementing the Runnable interface
- D. A thread is in "runnable thread" state, if and only if, the programmer calls the run() method from an object which is instance from a class that is derived from the Thread class
- E. A thread is in "runnable thread" state, if and only if, the programmer calls the stop() method from an object which is instance from a class that is derived from the Thread class

Răspunsul corect este **A**.

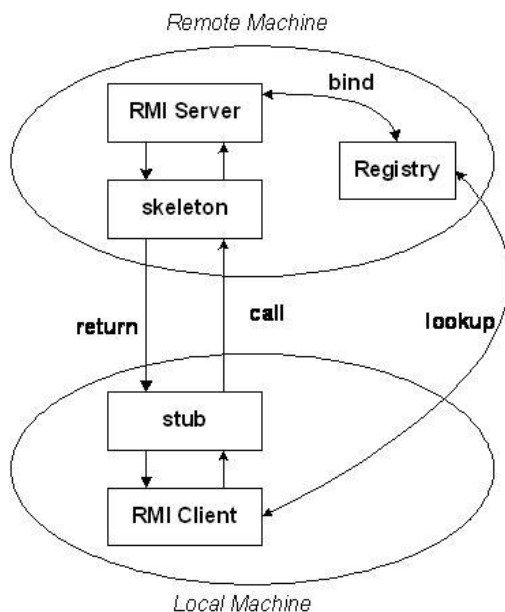
Explicație:

- **A. A thread is in „runnable thread” state, if and only if, the programmer calls the start() method from an object which is instance from a class that is derived from the Thread class**
 - Când se apelează metoda start() pe un obiect Thread, firul de execuție trece din starea NEW (nou) în starea RUNNABLE (executabil). În această stare, firul este eligibil pentru a fi executat de către schedulerul JVM. Aceasta este modalitatea corectă și principală de a iniția executarea unui fir de execuție într-un nou thread.
- **B. For a thread to be in "runnable thread” state, it is sufficient to create an object from a class that is derived from the Thread class**
 - Crearea unui obiect Thread îl pune doar în starea NEW. Nu îl face RUNNABLE.
- **C. For a thread to be in "runnable thread” state, it is sufficient to create an object from a class that is implementing the Runnable interface**
 - Implementarea interfeței Runnable definește codul care va fi executat de thread, dar nu creează și nu pornește un thread în sine. Trebuie să se

creeze un obiect Thread care să "înfășoare" acel Runnable și apoi să se apeleze start() pe obiectul Thread.

- **D. A thread is in "runnable thread" state, if and only if, the programmer calls the run() method from an object which is instance from a class that is derived from the Thread class**
 - Apelarea directă a metodei run() (e.g., myThread.run()) execută codul din metoda run() în firul de execuție *curent*, nu într-un nou fir de execuție separat. Prin urmare, obiectul myThread nu intră într-o stare RUNNABLE ca un thread independent.
- **E. A thread is in "runnable thread" state, if and only if, the programmer calls the stop() method from an object which is instance from a class that is derived from the Thread class**
 - Metoda stop() este deprecate și, atunci când este apelată, termină execuția unui thread, nu îl pune în starea RUNNABLE.

2. What is the true statement for *Java RMI 1.1* in terms of the logical flow actions' order:



- A. Sequence: 1. bind, 2. lookup, 3. call, 4. return
- B. Sequence: 1. call, 2. lookup, 3. bind, 4. return
- C. Sequence: 1. lookup, 2. call, 3. bind, 4. return
- D. Sequence: 1. lookup, 2. bind, 3. call, 4. return

E. Sequence: 1. call, 2. return, 3. bind, 4. lookup

Răspunsul corect este **A**.

Explicație:

Fluxul logic al acțiunilor în Java RMI (Remote Method Invocation) este următorul:

1. **Bind (server-side):** Serverul creează obiectul remote și îl înregistrează (îl "bind-uie") în RMI Registry sub un anumit nume. Acest pas trebuie să aibă loc *înainte* ca un client să poată găsi obiectul.
2. **Lookup (client-side):** Clientul contactează RMI Registry și efectuează un "lookup" pentru a obține o referință (un stub) la obiectul remote, folosind numele sub care a fost înregistrat de server.
3. **Call (client-side):** Odată ce clientul are stub-ul, poate apela metode pe obiectul remote ca și cum ar fi un obiect local. Stub-ul se ocupă de serializarea parametrilor și de trimiterea apelului către server.
4. **Return (server-side to client-side):** După execuția metodei pe server, rezultatul (sau o excepție) este serializat și trimis înapoi către client.

Prin urmare, secvența corectă este: **1. bind, 2. lookup, 3. call, 4. return.**

3. Knowing that at the offset 0x44 in the following SNMP traffic analysis is starting the OID – Object Identifier – value and knowing that the ASN.1 DER tag for OIDs declaration is 0x06:-Please specify which OID value is correct for the byte array sequence – the selected sequence: 0x30 0x0B 0x06 0x07 0x2B 0x06 0x01 0x02 0x01 0x01 0x03 0x05 0x00:

No. -	Time	Source	Destination	Protocol	Info
1	0.000000	10.2.5.146	10.2.4.198	SNMP	get-next-request SNMPv2-MIB::sysUpTime
2	0.000682	10.2.4.198	10.2.5.146	SNMP	get-response SNMPv2-MIB::sysUpTime.0

Frame 1 (81 bytes on wire, 81 bytes captured) Ethernet II, Src: Usi_43:1e:dc (00:16:41:43:1e:dc), Dst: Fujitsu_70:75:14 (00:17:42:70:75:14) Internet Protocol, Src: 10.2.5.146 (10.2.5.146), Dst: 10.2.4.198 (10.2.4.198) User Datagram Protocol, Src Port: nbx-au (2094), Dst Port: snmp (161) Simple Network Management Protocol version: version-1 (0) community: public data: get-next-request (1) get-next-request request-id: 1 error-status: noError (0) error-index: 0 variable-bindings: 1 item

- A. 1.3.6.1.2.1.1.3.5
- B. 1.3.6.1.2.1.1.3.5.0
- C. 1.2.1.1.3.5
- D. 1.3.6.1.2.1.1.3
- E. None

Pentru a decoda valoarea OID-ului din secvența de octeți, trebuie să urmăim regulile de codificare ASN.1 DER pentru Object Identifiers.

Secvența de octeți dată este: 0x30 0x0B 0x06 0x07 0x2B 0x06 0x01 0x02 0x01 0x01 0x03 0x05 0x00

Să o decodificăm pas cu pas:

- 0x30:** Acesta este *tag-ul* ASN.1 DER pentru SEQUENCE. Indică începutul unei secvențe.
- 0x0B:** Acesta este *lungimea* secvenței, adică 11 octeți.
- 0x06:** Acesta este *tag-ul* ASN.1 DER pentru OBJECT IDENTIFIER (așa cum este specificat și în întrebare). Indică faptul că urmează valoarea OID-ului.
- 0x07:** Acesta este *lungimea* valorii OID-ului, adică 7 octeți. Aceasta înseamnă că următorii 7 octeți reprezintă OID-ul.

Acum, să decodificăm cei 7 octeți ai valorii OID-ului: 0x2B 0x06 0x01 0x02 0x01 0x01 0x03

- **Primul octet (0x2B):** Primii doi sub-identificatori (sau noduri) ai unui OID sunt codificați în primul octet. Formula este $(X * 40) + Y$.
 - 0x2B în zecimal este 43.
 - Deci, $43 = (X * 40) + Y$.
 - Dacă luăm $X = 1$, atunci $Y = 43 - (1 * 40) = 3$.
 - Primii doi sub-identificatori sunt deci 1.3.
- **Următorii octeți:** Fiecare octet (sau grup de octeți) reprezintă un sub-identificator. Dacă bitul cel mai semnificativ (MSB) al unui octet este 1, înseamnă că sub-identificatorul continuă în octetul următor. Dacă MSB-ul este 0, octetul este ultimul pentru acel sub-identificator. Valoarea efectivă este formată prin concatenarea celor 7 biți mai puțin semnificativi.
 - **0x06:** MSB-ul este 0. Valoarea este 6. Următorul sub-identificator este 6.
 - **0x01:** MSB-ul este 0. Valoarea este 1. Următorul sub-identificator este 1.
 - **0x02:** MSB-ul este 0. Valoarea este 2. Următorul sub-identificator este 2.
 - **0x01:** MSB-ul este 0. Valoarea este 1. Următorul sub-identificator este 1.
 - **0x01:** MSB-ul este 0. Valoarea este 1. Următorul sub-identificator este 1.
 - **0x03:** MSB-ul este 0. Valoarea este 3. Următorul sub-identificator este 3.

Combinând toate sub-identificatorii, obținem OID-ul: 1.3.6.1.2.1.1.3.

Octeții 0x05 0x00 care urmează valorii OID-ului în secvența dată reprezintă un NULL ASN.1 (0x05 este tag-ul NULL, 0x00 este lungimea 0) și nu fac parte din OID-ul în sine, ci probabil dintr-o structură mai mare (cum ar fi o variabilă SNMP).

Comparând rezultatul cu opțiunile:

- A. 1.3.6.1.2.1.1.3.5
- B. 1.3.6.1.2.1.1.3.5.0
- C. 1.2.1.1.3.5
- D. 1.3.6.1.2.1.1.3
- E. None

Valoarea OID-ului corectă este **D. 1.3.6.1.2.1.1.3**.

4. For the Base 64 we have the following encoding table:-The „FTP” word has in ISO-8859-1 the following hex value: 0x46 0x54 0x50 – please specify what value is true for Base64:

- A. RL RQ = 0x11 0x25 0x11 0x10
- B. SLSQ = 0x12 0x0B 0x12 0x10
- C. None of the presented answers
- D. RLRQ = 0x11 0x0B 0x11 0x10
- E. SkSQ = 0x12 0x24 0x12 0x10

Pentru a codifica cuvântul "FTP" (valoare hex ISO-8859-1: 0x46 0x54 0x50) în Base64, vom urma pașii standard de codificare:

1. Conversia valorilor hex în binar:

- F = 0x46 = 01000110
- T = 0x54 = 01010100
- P = 0x50 = 01010000

2. Concatenarea șirurilor binare:

010001100101010001010000

3. Gruparea biților în blocuri de câte 6:

- 010001
- 100101
- 010001
- 010000

4. Conversia fiecărui bloc de 6 biți în valoarea sa zecimală:

- 010001 (binar) = 17 (zecimal)
- 100101 (binar) = 37 (zecimal)
- 010001 (binar) = 17 (zecimal)
- 010000 (binar) = 16 (zecimal)

5. Maparea valorilor zecimale la caracterele Base64 standard (RFC 4648):

(Deoarece tabelul nu a fost furnizat, folosim tabelul standard unde A=0, B=1,..., Z=25, a=26,..., z=51, 0=52,..., 9=61, +=62, /=63)

- 17 (zecimal) = **R**
- 37 (zecimal) = **l** (litera mică "L")
- 17 (zecimal) = **R**
- 16 (zecimal) = **Q**

Deci, cuvântul "FTP" codificat în Base64 este "RlRQ".

6. Conversia valorilor zecimale ale indicilor în hexazecimal:

- 17 (zecimal) = 0x11 (hex)
- 37 (zecimal) = 0x25 (hex)
- 17 (zecimal) = 0x11 (hex)
- 16 (zecimal) = 0x10 (hex)

Valoarea hexazecimală a indicilor este 0x11 0x25 0x11 0x10.

Comparând cu opțiunile date:

- A. RlRQ = 0x11 0x25 0x11 0x10
- B. SLSQ = 0x12 0x0B 0x12 0x10
- C. None of the presented answers
- D. RLRQ = 0x11 0x0B 0x11 0x10
- E. SkSQ = 0x12 0x24 0x12 0x10

Răspunsul calculat se potrivește exact cu opțiunea **A**.

Răspunsul corect este **A**.

5. Speaking about the development of a TOP-DOWN web service:

- A. the starting point is an already existing WSDL file**
- B. the WSDL file is created during web service development
- C. there is no such web service type
- D. there is no WSDL file involved

Răspunsul corect este **A**.

Explicație:

Dezvoltarea unui serviciu web de tip **TOP-DOWN** (numită și **WSDL-first**) înseamnă că procesul începe prin definirea contractului serviciului web, care este specificat într-un fișier WSDL (Web Services Description Language). Odată ce fișierul WSDL este stabilit, instrumente specifice sunt utilizate pentru a genera codul sursă (stub-uri și schelete) atât pentru implementarea serviciului pe partea de server, cât și pentru client.

- **A. the starting point is an already existing WSDL file** - Aceasta este definiția corectă a abordării TOP-DOWN. WSDL-ul este creat sau este deja existent *înainte* de a scrie codul de implementare.
- **B. the WSDL file is created during web service development** - Deși WSDL-ul este creat în timpul dezvoltării, această afirmație este prea generală. Poate fi adevărată și pentru abordarea Bottom-Up (unde WSDL-ul este generat din cod). În Top-Down, esențial este că *începi* cu WSDL-ul.
- **C. there is no such web service type** - Fals. Top-Down și Bottom-Up sunt abordări comune și bine definite în dezvoltarea serviciilor web.
- **D. there is no WSDL file involved** - Fals. Pentru serviciile web bazate pe SOAP, WSDL-ul este esențial pentru descrierea serviciului, indiferent de abordarea de dezvoltare.

6. Speaking about the development of a BOTTOM-UP web service:

A. there is no WSDL file involved

B. there is no such web service type

C. the WSDL file is created during web service development

D. the starting point is an already existing WSDL file

Răspunsul corect este **C**.

Explicație:

Dezvoltarea unui serviciu web de tip **BOTTOM-UP** (numită și **Code-first**) înseamnă că procesul începe prin scrierea codului de implementare a logicii de business a serviciului. După ce codul este scris, se utilizează instrumente (tool-uri) specifice pentru a genera fișierul WSDL (Web Services Description Language) din acest cod existent.

- **A. there is no WSDL file involved** - Fals. Pentru serviciile web bazate pe SOAP, un fișier WSDL este întotdeauna implicat, chiar dacă este generat automat.
- **B. there is no such web service type** - Fals. Bottom-Up și Top-Down sunt cele două abordări principale în dezvoltarea serviciilor web.
- **C. the WSDL file is created during web service development** - Aceasta este afirmația corectă. În abordarea Bottom-Up, WSDL-ul este generat *din codul existent* pe parcursul dezvoltării.

- **D. the starting point is an already existing WSDL file** - Aceasta descrie abordarea **TOP-DOWN (WSDL-first)**, nu Bottom-Up.

7. After the traffic analysis for FTP on port 21, we have the following network packet:-Please specify the true statement

```

Frame 20 (79 bytes on wire, 79 bytes captured)
Ethernet II, Src: IntelCor_cd:71:7f (00:13:e8:cd:71:7f), Dst: Giga-Byt_4a:6c:47 (00:14:85:4a:6c:47)
Internet Protocol, Src: 192.168.0.5 (192.168.0.5), Dst: 192.168.0.1 (192.168.0.1)
Transmission Control Protocol, Src Port: 1476 (1476), Dst Port: ftp (21), Seq: 39, Ack: 136, Len: 25
File Transfer Protocol (FTP)
  PORT 192,168,0,5,19,137\r\n
    Request command: PORT
    Request arg: 192,168,0,5,19,137

0000  00 14 85 4a 6c 47 00 13 e8 cd 71 7f 08 00 45 00  ...J]G.. ..q...E.
0010  00 41 1b 85 40 00 80 06 5d db c0 a8 00 05 c0 a8  .A..@... ].....
0020  00 01 05 c4 00 15 39 6e f4 05 7e 8c bd 19 50 18  ....9n ..~...P.
0030  b3 84 7d 71 00 00 50 4f 52 54 20 31 39 32 2c 31  ..}q..PO RT 192,1
0040  36 38 2c 30 2c 35 2c 31 39 2c 31 33 37 0d 0a 68,0,5,1 9,137..

```

- Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 137
- Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating the client TCP port 19137
- Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 19
- Perform a ”passive” FTP data transmission from the server (192.168.0.5) to the FTP client (192.168.0.1) by negotiation the server TCP port 19137
- Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 5001

Analizăm pachetul de rețea afișat:

- Rolurile Client-Server:**
 - **Src IP: 192.168.0.5, Src Port: 1476** (port efemer)
 - **Dst IP: 192.168.0.1, Dst Port: 21** (port standard FTP Control) Acest lucru indică faptul că 192.168.0.5 este **clientul FTP** și 192.168.0.1 este **serverul FTP**.
- Comanda FTP:**
 - Pachetul conține Request command: PORT.
 - Comanda PORT este folosită de **client** pentru a comunica **serverului** adresa IP și portul pe care clientul va asculta pentru o conexiune de date **intrată** de la server. Aceasta este caracteristica modului **FTP Activ**.
- Adresa IP și Portul negociat pentru conexiunea de date:**
 - Argumentul comenzii PORT este 192,168,0,5,19,137.
 - Primele patru numere (192,168,0,5) reprezintă adresa IP a mașinii pe care clientul va asculta conexiunea de date (în acest caz, chiar adresa clientului: 192.168.0.5).
 - Ultimele două numere (19,137) reprezintă portul TCP. Portul se calculează ca $(p1 * 256) + p2$.

- $p1 = 19$
- $p2 = 137$
- $\text{Portul} = (19 * 256) + 137 = 4864 + 137 = 5001.$

Deci, clientul (192.168.0.5) îi spune serverului (192.168.0.1) să stabilească o conexiune de date (în mod activ) înapoi către client, pe adresa 192.168.0.5 și portul 5001.

Acum să evaluăm opțiunile:

A. Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 137

* "Active" este corect. "negotiating with the client TCP port 137" este incorect (portul este 5001).

B. Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating the client TCP port 19137

* Portul 19137 este incorect.

C. Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 19

* Portul 19 este incorect.

D. Perform a “passive” FTP data transmission from the server (192.168.0.5) to the FTP client (192.168.0.1) by negotiation the server TCP port 19137

* "Passive" este incorect (este Active). 192.168.0.5 este clientul, nu serverul.

E. Carry out an “active” FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 5001

* "Active" este corect. Clientul (192.168.0.5) negociază cu serverul (192.168.0.1) portul TCP 5001 de pe client pentru transferul de date. Această afirmație descrie exact ce se întâmplă.

Răspunsul corect este **E**.

8. În urma rulării programului următor:

```
package eu.ase;
```

```
public class Certificate
```

```
{
```

```
    private int uid;
```

```
    private String issuerName;
```

```
    public Certificate() {
```

```
        this.uid = 0;
```

```
        this.issuerName = null;
```

```

}

public Certificate(int uidp, String issuerNamep) {
    this.uid = uidp;
    this.issuerName = issuerNamep;
}

public void setUid(int c) {
    this.uid = c;
}

public void setIssuerName(String n) {
    this.issuerName = n;
}

public String toString() {
    return new String("\tUid = "+this.uid+" , Issuer = "+this.issuerName);
}

} //end class Certificate

```

```

public class ProgMain
{
    public static void main(String args[]) {
        Certificate a = new Certificate(234, "Ion");
        Certificate b = new Certificate(355, "Vasile");

        b = a;

        b.setUid(888);
        b.setIssuerName("WWW");

        System.out.print("\n a="+a.toString());

        System.out.print("\n b="+b.toString());
    }
}

```

ce mesaje sunt afișate pe ecran?

- | | | |
|------------------------------|----|---------------------------|
| a. a=Uid = 888, Issuer = WWW | si | b=Uid = 888, Issuer = WWW |
| b. a=Uid = 234, Issuer = WWW | si | b=Uid = 888, Issuer = WWW |
| c. a=Uid = 234, Issuer = lon | si | b=Uid = 355, Issuer = WWW |
| d. a=Uid = 234, Issuer = lon | si | b=Uid = 888, Issuer = WWW |

Executăm pas cu pas programul Java:

1. Certificate a = new Certificate(234, "lon");

- Se creează un obiect Certificate cu uid = 234 și issuerName = "lon". Variabila a referențiază acest obiect.
- Stare: a -> {uid: 234, issuerName: "lon"}

2. Certificate b = new Certificate(355, "Vasile");

- Se creează un *alt* obiect Certificate cu uid = 355 și issuerName = "Vasile". Variabila b referențiază acest obiect nou.
- Stare:
 - a -> {uid: 234, issuerName: "lon"}
 - b -> {uid: 355, issuerName: "Vasile"}

3. b = a;

- **Acesta este pasul crucial.** Referința variabilei b este acum schimbată pentru a indica *același obiect* la care se referă variabila a. Obiectul inițial referențiat de b (cu uid=355, issuerName="Vasile") devine neaccesibil (și va fi colectat de garbage collector la un moment dat).
- Stare:
 - a -> {uid: 234, issuerName: "lon"}
 - b -> {uid: 234, issuerName: "lon"} (ambele referințe indică același obiect)

4. b.setUid(888);

- Deoarece b și a referențiază acum *același obiect*, această operație modifică câmpul uid al aceluia obiect partajat.
- Stare:

- a -> {uid: 888, issuerName: "lon"}
- b -> {uid: 888, issuerName: "lon"}

5. b.setIssuerName("WWW");

- Similar, această operație modifică câmpul issuerName al *aceluiși obiect* partajat.
- Stare:
 - a -> {uid: 888, issuerName: "WWW"}
 - b -> {uid: 888, issuerName: "WWW"}

6. System.out.print("\n a="+a.toString());

- Se va apela toString() pe obiectul referențiat de a.
- Output: \n a=\tUid = 888 , Issuer = WWW

7. System.out.print("\n b="+b.toString());

- Se va apela toString() pe obiectul referențiat de b. Deoarece b referențiază același obiect ca a, va afișa aceleași valori.
- Output: \n b=\tUid = 888 , Issuer = WWW

Rezultatul combinat pe ecran va fi:

a= Uid = 888 , Issuer = WWW

b= Uid = 888 , Issuer = WWW

Comparând cu opțiunile date:

- a. a=Uid = 888, Issuer = WWW si b=Uid = 888, Issuer = WWW
- b. a=Uid = 234, Issuer = WWW si b=Uid = 888, Issuer = WWW
- c. a=Uid = 234, Issuer = lon si b=Uid = 355, Issuer = WWW
- d. a=Uid = 234, Issuer = lon si b=Uid = 888, Issuer = WWW

Răspunsul corect este **a**.

9. Se consideră clasele:

```
class Baza{
    public int[] valori1;
```

```

    public Baza(){
        System.out.println("Apel CBI");
        valori1 = new int[5];
    }
    public Baza(int n){
        valori1 = new int[n];
        System.out.println("Apel CB");
    }
}

class Derivat extends Baza{
    public int[] valori2;
    public Derivat(int n){
        valori2 = new int[n];
        System.out.println("Apel CD");
    }
}

```

Execuția programului următor are ca efect

```

public class subiect2{
    public static void main(String args[]){
        Baza b = new Baza(5);
        Derivat d = new Derivat(6);
    }
}

```

a. Se obțin mesajele

Apel CBI;

Apel CB;

Apel CD;

b. Se obțin mesajele

Apel CB;

Apel CBI;

Apel CD;

c. Se obțin mesajele

Apel CD;

Apel CB;

d. Se obțin mesajele

Apel CB;

Apel CD;

Să analizăm execuția programului pas cu pas, ținând cont de regulile de apelare a constructorilor în Java, în contextul moștenirii.

Clasele și Constructorii:

- **class Baza:**
 - public Baza(): Constructor implicit (fără argumente). Afișează "Apel CBI".
 - public Baza(int n): Constructor cu argument. Afișează "Apel CB".
- **class Derivat extends Baza:**
 - public Derivat(int n): Constructor cu argument. Afișează "Apel CD".

Regula importantă pentru constructori în moștenire:

Când se creează o instanță a unei subclase, constructorul superclasei este întotdeauna apelat primul. Dacă constructorul subclasei nu include un apel explicit la `super()` (cu sau fără argumente), compilatorul inserează automat un apel la constructorul implicit (fără argumente) al superclasei (`super();`).

Execuția programului public class subiect2:

1. Baza b = new Baza(5);

- Acest apel invocă constructorul Baza(int n).
- Se execută corpul constructorului Baza(int n).
- Se afișează: Apel CB

2. Derivat d = new Derivat(6);

- Acest apel invocă constructorul Derivat(int n).
- **Deoarece constructorul Derivat(int n) nu are un apel explicit la super(), Java inserează automat super(); la începutul său.**
- Deci, prima dată se execută constructorul implicit al clasei Baza: Baza().
 - Se afișează: Apel CBI
- După ce constructorul superclasei se termină, se execută corpul constructorului Derivat(int n).
- Se afișează: Apel CD

Combinând toate mesajele în ordinea apariției lor:

1. Din new Baza(5): Apel CB
2. Din new Derivat(6) (apelul implicit super());: Apel CBI
3. Din new Derivat(6) (corpul constructorului Derivat): Apel CD

Rezultatul final este:

Apel CB

Apel CBI

Apel CD

Comparând cu opțiunile date:

a. Se obțin mesajele

Apel CBI;

Apel CB;

Apel CD;

b. Se obțin mesajele

Apel CB;

Apel CBI;

Apel CD;

c. Se obțin mesajele

Apel CD;

Apel CB;

d. Se obțin mesajele

Apel CB;

Apel CD;

Răspunsul corect este **b**.

Să analizăm execuția programului pas cu pas, ținând cont de regulile de apelare a constructorilor în Java, în contextul moștenirii.

Clasele și Constructorii:

- **class Baza:**
 - public Baza(): Constructor implicit (fără argumente). Afișează "Apel CBI".
 - public Baza(int n): Constructor cu argument. Afișează "Apel CB".
- **class Derivat extends Baza:**
 - public Derivat(int n): Constructor cu argument. Afișează "Apel CD".

Regula importantă pentru constructori în moștenire:

Când se creează o instanță a unei subclase, constructorul superclasei este întotdeauna apelat primul. Dacă constructorul subclasei nu include un apel explicit la `super()` (cu sau fără argumente), compilatorul inserează automat un apel la constructorul implicit (fără argumente) al superclasei (`super();`).

Execuția programului public class subiect2:

1. Baza b = new Baza(5);
 - Acest apel invocă constructorul Baza(int n).
 - Se execută corpul constructorului Baza(int n).
 - Se afișează: Apel CB
2. Derivat d = new Derivat(6);
 - Acest apel invocă constructorul Derivat(int n).
 - **Deoarece constructorul Derivat(int n) nu are un apel explicit la `super()`, Java inserează automat `super();` la începutul său.**

- Deci, prima dată se execută constructorul implicit al clasei Baza: Baza().
 - Se afișează: Apel CBI
- După ce constructorul superclasei se termină, se execută corpul constructorului Derivat(int n).
- Se afișează: Apel CD

Combinând toate mesajele în ordinea apariției lor:

1. Din new Baza(5): Apel CB
2. Din new Derivat(6) (apelul implicit super());: Apel CBI
3. Din new Derivat(6) (corpul constructorului Derivat): Apel CD

Rezultatul final este:

Apel CB

Apel CBI

Apel CD

Comparând cu opțiunile date:

a. Se obțin mesajele

Apel CBI;

Apel CB;

Apel CD;

b. Se obțin mesajele

Apel CB;

Apel CBI;

Apel CD;

c. Se obțin mesajele

Apel CD;

Apel CB;

d. Se obțin mesajele

Apel CB;

Apel CD;

Răspunsul corect este **b**.

10. Conceptul de polimorfism se implementeaza prin:

- a. Supraincarcare de functii si prin virtualizare
- b. Supraincarcare si derivare/mostenire de clase
- c. Supradefinire si virtualizare de functii
- d. Supraincarcare si supradefinire de functii

Conceptul de polimorfism în programarea orientată obiect se implementează prin două mecanisme principale:

1. **Polimorfism la compilare (Static Polymorphism):** Acesta este realizat prin **supraîncărcarea (overloading)** de funcții (metode). Aceasta înseamnă că poți avea mai multe metode cu același nume în aceeași clasă, dar cu liste de parametri diferite (număr, tip sau ordine). Compilatorul decide care metodă să apeleze în funcție de argumentele furnizate.
2. **Polimorfism la rulare (Dynamic Polymorphism):** Acesta este realizat prin **supradefinirea (overriding)** de funcții (metode) în contextul moștenirii de clase. O clasă derivată oferă propria implementare pentru o metodă care este deja definită în clasa sa de bază. Decizia privind care implementare a metodei să fie apelată (cea din clasa de bază sau cea din clasa derivată) se ia la momentul rulării programului, în funcție de tipul real al obiectului. Termenul de "virtualizare" (specific limbajului C++ prin virtual keyword) se referă la mecanismul care permite acestui polimorfism dinamic să funcționeze. În Java, metodele non-stactice, non-finale și non-private sunt "virtuale" implicit.

Analizând opțiunile:

- **a. Supraincarcare de functii si prin virtualizare:** Acoperă ambele forme de polimorfism, "virtualizare" referindu-se la mecanismul din spatele supradefinirii.
- **b. Supraincarcare si derivare/mostenire de clase:** Supraîncărcarea este corectă. Derivarea/moștenirea este o precondiție pentru polimorfismul la rulare, dar nu este în sine un mecanism *de implementare* a polimorfismului, ci mai degrabă o relație.
- **c. Supradefinire si virtualizare de functii:** Acoperă doar polimorfismul la rulare, ignorând supraîncărcarea.

- **d. Supraincarcare si supradefinire de functii:** Aceasta este cea mai precisă și completă descriere a modului în care polimorfismul este implementat, referindu-se direct la cele două tehnici cheie: supraîncărcarea (pentru polimorfismul static) și supradefinirea (pentru polimorfismul dinamic).

Ambele opțiuni **a** și **d** sunt puternice. Însă, **d** folosește termenii specifici operațiilor pe funcții care realizează polimorfismul (supraîncărcare și supradefinire), în timp ce "virtualizare" în a descrie mai degrabă proprietatea sau mecanismul intern al metodelor care permit supradefinirea. În contextul general al OOP, supraîncărcarea și supradefinirea sunt cele două concepte fundamentale pentru polimorfism.

Răspunsul cel mai precis este **d. Supraincarcare si supradefinire de functii.**

11. Se considera programul:

```
public class subiect4{

    public static void main(String args[]){

        int sir_valori[] = {10,15,2,56,67,5};

        try{

            RandomAccessFile raf = new RandomAccessFile("test.dat", "rw");

            for (int i = 0; i < sir_valori.length; i++)

                raf.writeInt(sir_valori[i]);

            raf.seek(12);

            System.out.println("Valoare gasita:" + raf.readInt());

            raf.close();

        }

        catch (IOException e){

            System.out.println("Eroare acces fisier");

        }

    }

}
```

Prin compilarea si executia programului se obtine:

Se afiseaza valoarea 2;

Se afiseaza valoarea 56;

Se afiseaza valoarea 67;

Se afiseaza valoarea 15;

Niciuna din valorile de mai sus

Programul scrie un șir de valori întregi într-un fișier binar (test.dat) folosind RandomAccessFile, apoi repoziționează pointerul de citire/scriere și citește o valoare.

Să analizăm pașii:

1. **int sir_valori[] = {10,15,2,56,67,5};**
 - Se inițializează un array de întregi.
2. **RandomAccessFile raf = new RandomAccessFile("test.dat", "rw");**
 - Se deschide (sau se creează) fișierul "test.dat" în modul de citire și scriere.
3. **for (int i = 0; i < sir_valori.length; i++) raf.writeInt(sir_valori[i]);**
 - Această buclă scrie fiecare element din sir_valori în fișier.
 - Metoda writeInt() scrie un întreg (Java int) ca 4 octeți.

Haiдеți să vedem ce valoare se află la ce offset:

- 10 ocupă octeții 0, 1, 2, 3
 - 15 ocupă octeții 4, 5, 6, 7
 - 2 ocupă octeții 8, 9, 10, 11
 - 56 ocupă octeții 12, 13, 14, 15
 - 67 ocupă octeții 16, 17, 18, 19
 - 5 ocupă octeții 20, 21, 22, 23
4. **raf.seek(12);**
 - Această linie mută pointerul de citire/scriere al fișierului la offset-ul de 12 octeți de la începutul fișierului.
 5. **System.out.println("Valoare gasita:" + raf.readInt());**
 - Metoda readInt() citește următorii 4 octeți de la poziția curentă a pointerului (care este 12) și îi interpretează ca pe un întreg.
 - Conform analizei de mai sus, valoarea care începe la offset-ul 12 este 56.

6. **raf.close();**

- Fișierul este închis.

Prin urmare, programul va afișa:

Valoare gasita:56

Comparând cu opțiunile:

- Se afiseaza valoarea 2;
- **Se afiseaza valoarea 56;**
- Se afiseaza valoarea 67;
- Se afiseaza valoarea 15;
- Niciuna din valorile de mai sus

Răspunsul corect este **Se afiseaza valoarea 56;**

12. What output may be generated by the following C/C++ OpenMP source code - be aware that there are 3 threads and the iterations from 0 to 4?

```
#include <stdlib.h>;
```

```
#include <stdio.h>;
```

```
#include "omp.h";
```

```
int main()
```

```
{
```

```
int nthreads, tid;
```

```
omp_set_num_threads(3);
```

```
int i;
```

```
#pragma omp parallel private(tid) shared(i)
```

```
{
```

```
tid = omp_get_thread_num();
```

```
printf("Hello world from (%d)\n", tid);
```

```

#pragma omp for
for(i = 0; i <= 4; i++)
{
    printf(" Iteration %d by (%d)", i, tid);
}
} //all threads join master thread and terminate

return 0;
}

```

A. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1) Iteration 5 by (1)

B. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1)

C. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1) Iteration 5 by (2)

D. Hello world from (0) Iteration 5 by (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0)

Să analizăm codul OpenMP pas cu pas:

1. **omp_set_num_threads(3);**: Programul este configurat să folosească 3 thread-uri. ID-urile acestor thread-uri vor fi 0, 1 și 2.
2. **#pragma omp parallel private(tid) shared(i);**: Aceasta definește o regiune paralelă.
 - Fiecare dintre cele 3 thread-uri va executa codul din interiorul acestei regiuni.
 - Variabila tid este private, adică fiecare thread va avea propria sa copie a variabilei tid.
 - Variabila i este shared, ceea ce înseamnă că toate thread-urile vor lucra cu aceeași variabilă i. (Totuși, pentru bucla for paralelă, variabila de iterație (i) este implicit privatizată pentru calculul iterațiilor de către fiecare thread, prevenind condițiile de cursă pe contor. shared(i) în contextul #pragma omp parallel înseamnă că i este o variabilă partajată în

regiunea paralelă, dar directiva #pragma omp for are reguli specifice pentru variabilele de control ale buclei.)

3. **printf("Hello world from (%d)\n", tid);:**

- Fiecare dintre cele 3 thread-uri va afișa acest mesaj o singură dată.
- Ordinea în care aceste mesaje apar este **nedeterminată** (depinde de scheduler-ul sistemului de operare și de când fiecare thread ajunge să execute această linie). Putem vedea (0), (1), (2) sau (2), (0), (1) etc.

4. **#pragma omp for:**

- Această directivă împarte iterațiile buclei for între cele 3 thread-uri.
- Bucloa for(i = 0; i <= 4; i++) va executa 5 iterații (i ia valorile 0, 1, 2, 3, 4).
- Fiecare dintre aceste 5 iterații va fi executată de unul dintre cele 3 thread-uri. Distribuția exactă a iterațiilor între thread-uri este, de asemenea, **nedeterminată** (depinde de politica de scheduling, care e de obicei static implicit, dar chiar și așa, ordinea de execuție a printf-urilor dintre thread-uri va fi aleatorie).
- Este crucial că nu va exista nicio iterație cu i = 5 sau mai mare, deoarece bucla se oprește la i = 4.

5. **printf(" Iteration %d by (%d)", i, tid);:**

- Această linie va fi executată de 5 ori în total (o dată pentru fiecare iterație a buclei for).
- i va fi valoarea iterației curente (0, 1, 2, 3, sau 4).
- tid va fi ID-ul thread-ului care execută acea iterație (0, 1, sau 2).

Caracteristicile unei ieșiri valide:

- Vor exista exact 3 mesaje "Hello world from (X)", câte unul pentru fiecare ID de thread (0, 1, 2). Ordinea lor poate varia.
- Vor exista exact 5 mesaje "Iteration %d by (%d)", pentru valorile i de la 0 la 4.
- Nicio valoare i nu va fi mai mare decât 4.
- tid în mesajele "Iteration" va fi unul dintre 0, 1 sau 2.
- Ordinea generală a mesajelor (între "Hello world" și "Iteration", și între iterații) va fi interconectată/dezordonată.

Să verificăm opțiunile:

- **A. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1) Iteration 5 by (1)**
 - Conține "Iteration 3 by (1)" și "Iteration 5 by (1)". (1) nu este un ID de thread valid (trebuie să fie 0, 1 sau 2). De asemenea, "Iteration 5" este incorectă.
- **B. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1)**
 - Conține 3 mesaje "Hello world" (de la thread-urile 0, 2, 1) – ordine validă, aleatorie.
 - Conține 5 mesaje "Iteration" pentru $i = 0, 1, 2, 3, 4$. Toate valorile i sunt valide și toate ID-urile de thread sunt valide (0, 1, 2). Ordinea este interconectată, ceea ce este normal.
 - Aceasta este o ieșire posibilă și validă.
- **C. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1) Iteration 5 by (2)**
 - Conține "Iteration 5 by (2)", care este o iterație invalidă (bucula merge doar până la $i=4$).
- **D. Hello world from (0) Iteration 5 by (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0)**
 - Conține "Iteration 5 by (0)", care este o iterație invalidă.
 - De asemenea, lipsesc mesaje "Hello world" (doar 2 din 3 thread-uri sunt prezente).

Prin urmare, singura opțiune care respectă toate regulile de execuție OpenMP este **B**.

13. What statement is true for the following C/C++ OpenMP source code?

```
#include <iostream>
```

```
#include <vector>
```

```
#include <omp.h>
```

```
using namespace std;
```

```

#include <time.h>

clock_t start, stop;

int main(int argc, char** argv)
{
    int nr_elements = 10000000, i = 0, nrThreads = 4;
    vector<int> u1(nr_elements, 1), u2(nr_elements, 2);
    vector<int> result(nr_elements, 0); vector<int> resultmp(nr_elements, 0);
    //nrThreads = atoi(argv[1]);

    start = clock();

    for (i = 0; i < nr_elements; i++)
    {
        result[i] = u1[i] + u2[i];
    }

    stop = clock();

    for (int i = 0; i < 20; i++)
        cout << " , " << result[i];

    cout << "It took " << (stop - start) << " clicks (" << (((float)(stop - start)) /
CLOCKS_PER_SEC) << " seconds)." << endl;

    // a horksharig for construct to add vectors:
    omp_set_num_threads(nrThreads);

    start = clock();

#pragma omp parallel for

```

```

for (i = 0; i < nr_elements; i++)
{
    resulttmp[i] = u1[i] + u2[i];
}

stop = clock();

cout << "It took " << (stop - start) << " clicks (" << (((float)(stop - start)) /
CLOCKS_PER_SEC) << " seconds)." << endl;

for (int j = 0; j < 20; j++)
    cout << " , " << resulttmp[j];

return 0;
} //end main

```

The program is adding 2 arrays/Vectors using 2 threads

The program is adding 4 arrays/vectors using 4 threads

The program is not using OpenMP syntax

The program is adding 4 arrays/vectors using 4 threads

None of the existing answers

Să analizăm programul C/C++ OpenMP pentru a identifica afirmația corectă:

1. Inițializarea variabilelor:

- `int nr_elements = 10000000;` - Dimensiunea vectorilor este de 10 milioane de elemente.
- `int nrThreads = 4;` - Se specifică numărul dorit de thread-uri la **4**.
- `vector<int> u1(nr_elements, 1), u2(nr_elements, 2);` - Se inițializează două vectori, u1 cu valoarea 1 și u2 cu valoarea 2.
- `vector<int> result(nr_elements, 0); vector<int> resulttmp(nr_elements, 0);` - Doi vectori pentru stocarea rezultatelor.

2. Prima secțiune de calcul (secvențială):

- Codul efectuează $result[i] = u1[i] + u2[i]$; într-o buclă for simplă. Aceasta adună elementele a **două** vectori ($u1$ și $u2$). Această secțiune **nu** folosește OpenMP.

3. Configurarea OpenMP:

- `omp_set_num_threads(nrThreads);` - Această linie setează numărul de thread-uri pentru regiunile paralele următoare la valoarea variabilei `nrThreads`, care este **4**.

4. A doua secțiune de calcul (paralelă OpenMP):

- `#pragma omp parallel for` - Această directivă indică faptul că bucla for care urmează va fi executată în paralel de mai multe thread-uri. Numărul de thread-uri va fi **4**, conform setării anterioare.
- Codul din buclă este $resultmp[i] = u1[i] + u2[i]$; Aceasta adună din nou elementele a **două** vectori ($u1$ și $u2$).

Evaluarea afirmațiilor:

- **The program is adding 2 arrays/Vectors using 2 threads**
 - Afirmația "adding 2 arrays/Vectors" este corectă.
 - Afirmația "using 2 threads" este **incorectă**, deoarece programul folosește 4 thread-uri (`nrThreads = 4`, `omp_set_num_threads(nrThreads)`).
- **The program is adding 4 arrays/vectors using 4 threads**
 - Afirmația "adding 4 arrays/vectors" este **incorectă**, deoarece programul adună doar 2 vectori ($u1$ și $u2$).
 - Afirmația "using 4 threads" este corectă pentru secțiunea paralelă.
- **The program is not using OpenMP synthax**
 - Această afirmație este **incorectă**. Programul include `omp.h` și folosește directivele OpenMP (`omp_set_num_threads`, `#pragma omp parallel for`).
- **The program is adding 4 arrays/vectors using 4 threads**
 - Această afirmație este o duplicare a celei de-a doua și este, de asemenea, **incorectă** (din același motiv).
- **None of the existing answers**

- Deoarece niciuna dintre celelalte afirmații nu este complet adevărată, această opțiune este cea corectă. Programul adună două vectori și utilizează 4 thread-uri pentru partea paralelă.

Răspunsul corect este **None of the existing answers.**

14. What statement is true taking into account the following Java EE source code in terms of JMS -Java Messaging Service API?

```
package eu.ase.jms;

import javax.jms.x;
import javax.naming.x;

public class QueueReceiver{

    public static void main(String[] args){

        String queueName = null;

        Context      jndiContext= null;

        QueueConnectionFactory queueConnectionFactory= null; QueueConnection
            queueConnection = null;

        QueueSession      queueSession = null;

        Queuequeue = null;

        QueueReceiver      queueReceiver= null;

        TextMessage message = null;

        queueName = new String(queue/testQueue);

        by{

            jndiContext= new InitialContext();

        } catch (NamingException e){

            System.outprintln("Could not create JNDI API" + "context:" + e.toString());

        }

        try {

            queueConnectionFactory=
            (QueueConnectionFactory)jndiContext.lookup('UILConnectionFactory');
```

```

queue = (Queue) jndiContext.lookup(queueName);
} catch (NamingException e){
System.out.println('JNDI API lookup failed:" + e.toString0):
System.exit(1);
}
111,{
queueConnection = queueConnectionFactory.createQueueConnection0;
queueSession = queueConnection.createQueueSession(false.
Session.AUTO_ACKNOWLEDGE);
queueReceiver= queueSession.createReceiver(queue);
queueConnection.starta
while (true)(
Message m = queueReceiver.receive(1);
if (m != null)(
if (m instanceof TextMessage)(
message = (TextMessage) m;
System.outprintln('Reading message:" + message.getTexlQ);
} catch (NamingException e)(
System.out.println("JNDI API lookup failed:" + e.toSt(ing0):
System.exit(1);
}
try
queueConnection = queueConnectionFactoryvcreateQueueConnection0;
queueSession = queueConnection.createQueueSession(false.
Sessionhttps://docs.google.com/document/d/1iSHHrGgFDNWpowUZs74GP8OPZvK9iiW-tucn8VWz4e0/edit?usp=drive\_web('Reading message:" + message.getText0);
} else (
break;
}

```

```

}
}
} catch (JMSEException e){
System.out.println("Exception occurred:" + e.toString0);
}finally(
if (queueConnection != null){
try
queueConnedion.close();
} catch (JMSEException e){}
}end main }
//end class

```

The client program consumes/receives messages from a Java EE server queue and it is looping forever.

The client program consumes/receives messages from a Java EE server queue and it is breaking the while loop if and only if it receives a text message

The source code is not compliant with JMS API

None of the existing answers

The client program consumes/receives messages from a Java EE server queue and it is breaking the while loop if and only if it receives a non-text message.

Analizând codul sursă furnizat pentru QueueReceiver, putem observa mai multe probleme și putem deduce comportamentul intenționat:

1. **Sintaxă și erori:** Codul conține numeroase erori de sintaxă (ex: createQueueConnection0 în loc de createQueueConnection(), Session.AUTO_ACKNOWLEDGE în loc de Session.AUTO_ACKNOWLEDGE, starta în loc de start(), getTexlQ în loc de getText(), paranteze și blocuri try-catch incorect plasate/închise, etc.). Din cauza acestor erori, codul **nu este compilabil și, prin urmare, nu este "compliant" (conform) cu sintaxa Java și, implicit, cu modul de utilizare al API-ului JMS.**
2. **Logica de primire a mesajelor (presupunând o corectare a erorilor de sintaxă):**

- Linia Message m = queueReceiver.receive(1); indică faptul că se încearcă primirea unui mesaj cu un timeout de 1 milisecundă. Aceasta înseamnă că apelul nu va bloca pe termen nelimitat.
- Blocul if (m != null) verifică dacă un mesaj a fost primit în intervalul de timeout.
 - În interiorul acestui bloc, if (m instanceof TextMessage) verifică dacă mesajul este de tip TextMessage. Dacă da, conținutul său este afișat.
 - Blocul else corespunzător (else pentru if (m instanceof TextMessage)) conține break;. Aceasta înseamnă că, dacă se primește un mesaj, dar acesta **NU** este un TextMessage, bucla while este întreruptă.
- Blocul else corespunzător lui if (m != null) conține de asemenea break;. Aceasta înseamnă că, dacă **NU** se primește niciun mesaj în intervalul de 1 milisecundă (adică m este null), bucla while este întreruptă.

Concluzie privind logica buclei while:

Bucla while(true) se întrerupe în următoarele situații:

- Dacă nu se primește niciun mesaj în 1 milisecundă (m == null).
- Dacă se primește un mesaj, dar acesta nu este de tip TextMessage.

Deci, bucla **NU** rulează la infinit.

Evaluarea afirmațiilor:

- **The client program consumes/receives messages from a Java EE server queue and it is looping forever.**
 - **Fals.** Bucla are condiții de întrerupere.
- **The client program consumes/receives messages from a Java EE server queue and it is breaking the while loop if and only if it receives a text message**
 - **Fals.** Se întrerupe dacă **NU** primește un text message (sau niciun mesaj).
- **The source code is not compliant with JMS API**
 - Această afirmație este **Adevărată**. Datorită numeroaselor erori de sintaxă menționate anterior, codul nu este conform cu sintaxa Java necesară pentru a utiliza corect API-ul JMS și, prin urmare, nu ar compila sau rula.
- **None of the existing answers**

- Deși ar putea fi considerată o opțiune dacă am ignora erorile de sintaxă și am căuta o descriere perfectă a logicii, prezența erorilor de sintaxă face ca afirmația "not compliant" să fie direct adevărată.
- **The client program consumes/receives messages from a Java EE server queue and it is breaking the while loop if and only if it receives a non-text message.**
 - **Fals.** Se întrerupe dacă primește un non-text message SAU dacă nu primește niciun mesaj. "If and only if" face afirmația incorectă.

Prin urmare, cea mai precisă și incontestabilă afirmație adevărată despre codul sursă *așa cum este prezentat* este că nu este conform (compilabil/executabil) cu API-ul JMS din cauza erorilor de sintaxă.

Răspunsul corect este: **The source code is not compliant with JMS API**

15. What statement is true with respect to the following Java EE - RESTful Service class source code (using Jersey API):

```
package eu.ase.rest;

import javax.ws.rs.GET; import javax.ws.rs.Path; import javax.ws.rs.Produces;

import javax.ws.rs.core.MediaType;

@Path("UserService") public class UserService (

    UserDao userDao = new UserDao();

    @GET @Path("/users")

    @Produces(MediaType.APPLICATION_XML)
    // @Produces(MediaType.APPLICATION_JSON) public List<User> getUsers()

    return userDao.getAllUsers();

}

// End class
```

- A. It is a RESTful class which handles HTTP GET requests for getUsers resource
- B. None of the existing answers
- C. It is not a correct Java source code
- D. It is a RESTful class which handles both HTTP GET and POST requests for getUsers resource

E. It is a RESTful class which handles both HTTP POST requests for getUsers resource

Analizăm codul sursă furnizat pentru clasa UserService:

1. **Importuri:** Importurile javax.ws.rs.GET, javax.ws.rs.Path, javax.ws.rs.Produces, javax.ws.rs.core.MediaType indică clar că această clasă utilizează API-ul JAX-RS pentru a defini un serviciu RESTful.
2. **Anotarea la nivel de clasă:**
 - @Path("UserService"): Această anotație definește calea de bază (URI-ul de bază) pentru acest serviciu. Toate resursele din această clasă vor fi accesibile sub calea /UserService.
3. **Metoda getUsers():**
 - @GET: Această anotație este crucială. Ea indică faptul că metoda getUsers() va răspunde la cererile HTTP de tip **GET**.
 - @Path("/users"): Aceasta specifică o sub-cale pentru metodă. Combinată cu calea de bază a clasei, resursa completă pentru această metodă va fi /UserService/users.
 - @Produces(MediaType.APPLICATION_XML): Aceasta indică faptul că metoda va returna date în format XML.
 - public List<User> getUsers(): Semnătura metodei sugerează că va returna o listă de obiecte User.
 - return userDao.getAllUsers(); (cu o corectură de tipar getAllUsers()): Acesta este un apel către un obiect UserDao, indicând că metoda obține toate datele despre utilizatori.

Evaluarea afirmațiilor:

- **A. It is a RESTful class which handles HTTP GET requests for getUsers resource**
 - Este o clasă marcată cu @Path și conține o metodă marcată cu @GET și @Path("/users"), indicând clar că este o clasă RESTful care gestionează cereri HTTP GET pentru resursa "users" (sau "getUsers" așa cum este numită metoda). Această afirmație este **adevărată**.
- **B. None of the existing answers**
 - Deoarece am găsit o afirmație adevărată (A), aceasta este falsă.
- **C. It is not a correct Java source code**

- Deși există erori minore de sintaxă (UserDao0 în loc de new UserDao(), getAllUsers0 în loc de getAllUsers()), structura fundamentală și utilizarea anotațiilor JAX-RS sunt conceptual corecte. Într-un context de întrebare despre natura unui serviciu RESTful, accentul este pe design și funcționalitate, nu pe greșeli de tipar. Deci, din punct de vedere funcțional și de design, este o clasă RESTful corectă. Dacă se interpretează strict ca "va compila fără erori", atunci ar fi falsă, dar întrebarea se referă la "ce afirmație este adevărată ... în termeni de... RESTful Service class".
- **D. It is a RESTful class which handles both HTTP GET and POST requests for getUsers resource**
 - Metoda getUsers() este marcată doar cu @GET. Nu există nicio anotație @POST. Această afirmație este **falsă**.
- **E. It is a RESTful class which handles both HTTP POST requests for getUsers resource**
 - Metoda getUsers() este marcată doar cu @GET. Această afirmație este **falsă**.

Prin urmare, cea mai precisă afirmație despre natura și funcționalitatea acestei clase este că este un serviciu RESTful care gestionează cereri HTTP GET pentru resursa specificată.

Răspunsul corect este **A. It is a RESTful class which handles HTTP GET requests for getUsers resource.**

16. SUN Java CORBA has 3 parts:

A. None of the existing answers

B. — ORB (Object Request Broker) in Class/JAR

— Naming Service: rmiregistry for both Non-persistent & Persistent

— IDL idltojava & javatoidl compiler— now: idlj.exe

C. — ORB (Object Request Broker) in Class/JAR

— Naming Service: COS — CORBA Object Service: tnameserv.exe (Non-persistent) & orbd.exe (Persistent)

— IDL idltojava & javatoidl compiler— now: idlj.exe

D. — ORB (Object Request Broker) as OS Service/Process

— Naming Service: COS — CORBA Object Service: tnameserv.exe (Non-persistent) & orbd.exe (Persistent)

— IDL idltojava & javatoidl compiler— now: idlj.exe

E. — ORB (Object Request Broker) as OS Service/Process

— Naming Service: COS — CORBA Object Service: only orbd.exe

— IDL idltojava & javatoidl compiler— now: idlj.exe

Implementarea CORBA în Java de la SUN (cunoscută sub numele de Java IDL) este compusă, în general, din următoarele trei părți principale:

1. **ORB (Object Request Broker - Broker de cereri de obiecte):** Acesta este "motorul" CORBA, care permite obiectelor distribuite să comunice transparent. În Java IDL, ORB-ul este furnizat ca un set de clase Java (e.g., în pachetul org.omg.CORBA) și este inclus în JRE/JDK, fiind instanțiat în cadrul aplicației Java, nu ca un serviciu de sistem de operare separat.
2. **Naming Service (Serviciu de Nume):** Obiectele CORBA sunt înregistrate și căutate prin intermediul unui Serviciu de Nume. Standardul CORBA definește **COS Naming Service**. Sun Java IDL oferă două implementări principale pentru acest serviciu:
 - tnameserv.exe: Un serviciu de nume tranzitoriu (non-persistent). Obiectele înregistrate se pierd la oprirea serviciului.
 - orbd.exe: Un Object Request Broker Daemon care oferă și un serviciu de nume persistent, capabil să păstreze referințele la obiecte chiar și după repornire.
3. **IDL Compilers (Compilatoare IDL):** CORBA folosește IDL (Interface Definition Language) pentru a descrie interfețele obiectelor într-un mod independent de limbaj. Sunt necesare compilatoare pentru a traduce definițiile IDL în cod specific limbajului (stubs pentru clienți și skeletons pentru servere).
 - Sun a oferit instrumentul idltojava (o variantă mai veche) și, ulterior, l-a consolidat și îmbunătățit sub forma idlj.exe.

Să analizăm opțiunile date:

- **A. None of the existing answers**
 - Se va determina după evaluarea celorlalte opțiuni.

- **B. — ORB (Object Request Broker) in Class/JAR — Naming Service: rmiregistry for both Non-persistent & Persistent — IDL idltojava &javatoidl compiler— now: idlj.exe**
 - ORB-ul în Class/JAR este corect.
 - **Incorrect:** rmiregistry este serviciul de nume pentru Java RMI (Remote Method Invocation), nu pentru CORBA.
- **C. — ORB (Object Request Broker) in Class/JAR — Naming Service: COS — CORBA Object Service: tnameserv.exe (Non-persistent) & orbd.exe (Persistent) — IDL idltojava &javatoidl compiler— now: idlj.exe**
 - ORB-ul în Class/JAR este corect.
 - Serviciul de Nume COS cu tnameserv.exe (non-persistent) și orbd.exe (persistent) este corect.
 - Compilatorul IDL idlj.exe (cu mențiunea uneltelor anterioare idltojava/javatoidl) este corect.
 - Această opțiune este **corectă**.
- **D. — ORB (Object Request Broker) as OS Service/Process — Naming Service: COS — CORBA Object Service: tnameserv.exe (Non-persistent) & orbd.exe (Persistent) — IDL idltojava &javatoidl compiler— now: idlj.exe**
 - **Incorrect:** ORB-ul din Java IDL nu rulează ca un serviciu/proces de sistem de operare independent, ci este instanțiat ca un obiect în cadrul unei aplicații Java.
- **E. — ORB (Object Request Broker) as OS Service/Process — Naming Service: COS — CORBA Object Service: only orbd.exe — IDL idltojava &javatoidl compiler— now: idlj.exe**
 - **Incorrect:** ORB-ul ca OS Service/Process este greșit.
 - **Incorrect:** Nu este "only orbd.exe"; tnameserv.exe există și el.

Pe baza analizei, opțiunea C descrie cel mai precis cele trei componente majore ale implementării SUN Java CORBA (Java IDL).

Răspunsul corect este **C**.

17. What statement is not true for OpenMP programming model?

A. OpenMP programming model uses Explicit Parallelism: Programmer has full control over parallelization. OpenMP is not an automatic parallel programming model.

- B. OpenMP programming model uses shared memory, thread-based parallelism: OpenMP is based on the existence of multiple threads in the shared memory programming paradigm. (A shared memo)
- C. OpenMP programming model is designed to guarantee that input or output to the same file is synchronous when executed in parallel.
- D. OpenMP programming model uses Compiler directive based: Most OpenMP parallelism is specified through the use of compiler directives which are embedded in the source code.
- E. None of the existing answers

Afirmația care **NU este adevărată** pentru modelul de programare OpenMP este:

- **C. OpenMP programming model is designed to guarantee that input or output to the same file is synchronous when executed in parallel.**

Explicație:

OpenMP **nu garantează** în mod implicit sincronizarea operațiilor de intrare/ieșire (I/O) la același fișier atunci când sunt executate în paralel. Dimpotrivă, dacă mai multe thread-uri încearcă să scrie concurent în același fișier fără mecanisme de sincronizare explicite (cum ar fi secțiuni critice, mutex-uri, sau folosirea unor biblioteci I/O thread-safe), se pot produce condiții de cursă (race conditions) și date corupte sau intercalate în fișier. Programatorul este responsabil pentru asigurarea siguranței concurenței la accesarea resurselor partajate, inclusiv a fișierelor.

Să analizăm și celelalte afirmații, care sunt adevărate despre OpenMP:

- **A. OpenMP programming model uses Explicit Parallelism: Programmer has full control over parallelization. OpenMP is not an automatic parallel programming model.**
 - **Adevărat.** Programatorii trebuie să insereze explicit directive OpenMP (`#pragma omp ...`) în codul sursă pentru a indica zonele paralele și distribuția muncii. Nu paralelizează codul automat.
- **B. OpenMP programming model uses shared memory, thread-based parallelism: OpenMP is based on the existence of multiple threads in the shared memory programming paradigm. (A shared memo)**
 - **Adevărat.** OpenMP este conceput pentru arhitecturi cu memorie partajată, unde mai multe thread-uri operează pe același spațiu de memorie.

- **D. OpenMP programming model uses Compiler directive based: Most OpenMP parallelism is specified through the use of compiler directives which are embedded in the source code.**
 - **Adevărat.** Metoda principală de specificare a paralelismului în OpenMP este prin directivele de compilator (cunoscute și sub numele de "pragma").

Prin urmare, afirmația falsă este C.

18. What statement is true for OpenMP?

- A. None of the existing answers.
- B. OpenMP is implemented identically by all vendors
- C. OpenMP is meant for distributed-memory parallel systems
- D. OpenMP is guaranteed to make the most efficient use of shared memory
- E. OpenMP is meant to cover compiler-generated automatic parallelization and directives to the compiler to assist such parallelization

Să analizăm fiecare afirmație despre OpenMP:

- **A. None of the existing answers.**
 - Aceasta este o posibilitate dacă toate celelalte afirmații sunt false.
- **B. OpenMP is implemented identically by all vendors.**
 - **Fals.** Deși OpenMP este un standard (o specificație API), implementările sale concrete (de către diverși producători de compilatoare precum GCC, Intel, Microsoft Visual C++) pot varia în detalii de optimizare, performanță și comportament implicit (ex: strategii de scheduling), chiar dacă respectă specificațiile standard.
- **C. OpenMP is meant for distributed-memory parallel systems.**
 - **Fals.** OpenMP este conceput în mod fundamental pentru sisteme paralele cu **memorie partajată (shared-memory systems)**. Pentru sistemele cu memorie distribuită, modele precum MPI (Message Passing Interface) sunt utilizate în mod obișnuit.
- **D. OpenMP is guaranteed to make the most efficient use of shared memory.**

- **Fals.** OpenMP oferă instrumente pentru paralelism, dar nu garantează cea mai eficientă utilizare a memoriei partajate. Eficiența depinde în mare măsură de modul în care programatorul structurează algoritmul paralel, gestionează localitatea datelor, minimizează false sharing-ul și asigură sincronizarea corectă. Un cod OpenMP scris prost poate fi mai puțin eficient decât unul secvențial.
- **E. OpenMP is meant to cover compiler-generated automatic parallelization and directives to the compiler to assist such parallelization.**
 - **Fals.** OpenMP nu este un model de paralelism automat generat de compilator. El este un model de paralelism **explicit**, bazat pe directive. Programatorul introduce explicit directive (`#pragma omp ...`) în cod pentru a indica compilatorului cum să paralelizeze anumite secțiuni. Deși directivele "asistă" compilatorul, OpenMP în sine nu "acoperă" sau nu este o formă de paralelism automat.

Concluzie: Toate afirmațiile B, C, D și E sunt false. Prin urmare, singura afirmație adevărată este că niciuna dintre opțiunile prezentate nu este corectă.

Răspunsul corect este **A. None of the existing answers.**

19. Which statement is NOT true for CORBA - Common Object Request Broker Architecture - in terms of the concepts?

- A. In CORBA is mandatory to have both the clients and servants in Java.
- B. None of the existing answers
- C. CORBA Skeleton lives on server
- D. CORBA Skeleton receives requests from stub via ORB
- E. CORBA Stub lives on client

Afirmația care **NU este adevărată** despre CORBA (Common Object Request Broker Architecture) este:

- **A. In CORBA is mandatory to have both the clients and servants in Java.**

Explicație:

Unul dintre principiile fundamentale ale CORBA este **independența de limbaj**. CORBA a fost conceput pentru a permite interoperabilitatea între obiecte distribuite scrise în limbaje de programare diferite. Astfel, un client poate fi implementat în C++, iar servant-ul (obiectul server) în Java, sau invers, sau în orice alt limbaj pentru care există o mapare IDL. Nu este deloc obligatoriu ca atât clienții, cât și servant-urile să fie în Java.

Să verificăm celelalte afirmații, care sunt adevărate despre CORBA:

- **C. CORBA Skeleton lives on server**
 - **Adevărat.** Skeleton-ul este codul de pe partea serverului. El primește cererile de la ORB (Object Request Broker) și le direcționează către implementarea efectivă a obiectului (servant-ul).
- **D. CORBA Skeleton receives requests from stub via ORB**
 - **Adevărat.** Clientul apelează o metodă pe stub-ul său local. Stub-ul împachetează (marshals) cererea și o trimite prin rețea către ORB. ORB-ul o rutează către server, unde skeleton-ul o primește de la ORB.
- **E. CORBA Stub lives on client**
 - **Adevărat.** Stub-ul este codul de pe partea clientului. El acționează ca o reprezentare locală (un proxy) a obiectului remote, gestionând detaliile comunicării la distanță.

Prin urmare, afirmația falsă este **A**.

20. If the number of OpenMP threads is set to 4 (OMP_NUM_THREAD = 4). What would be a potential, output of the following Open MP Parallel source code:

```
include <stdlib.h>
```

```
include <stdio.h>
```

```
include <omp.h>
```

```
int main ()
```

```
{
```

```
    pragma omp parallel
```

```
{
```

```
    int ID = omp_get_thread_num();
```

```
    printf("Hello (%d)\n", ID);
```

```
    printf("World (%d)\n", ID);
```

```
}
```

```
return 0;
```

}

A.

Hello (0)

World (0)

Hello (2)

World (2)

Hello (1)

World (1)

Hello (2)

World (2)

B.

Hello (0)

World (0)

Hello (0)

World (1)

Hello (1)

World (2)

Hello (2)

World (3)

C.

Hello (0)

world (1)

Hello (2)

world (3)

D.

None of the existing answers

E.

Hello (0)

World (0)

Hello (2)

World (2)

Hello (1)

World (1)

Hello (3)

World (3)

Să analizăm programul OpenMP pas cu pas și să determinăm proprietățile ieșirii:

1. **omp_set_num_threads(4);**: Această linie (presupunând că a fost adăugată sau că variabila de mediu OMP_NUM_THREADS este setată la 4) indică faptul că se vor crea 4 thread-uri. ID-urile acestor thread-uri vor fi 0, 1, 2 și 3.
2. **#pragma omp parallel**: Această directivă creează o regiune paralelă. Fiecare dintre cele 4 thread-uri va executa codul din interiorul acestui bloc.
3. **int ID = omp_get_thread_num();**: Fiecare thread își obține ID-ul unic (0, 1, 2 sau 3) și îl stochează într-o variabilă ID privată.
4. **printf("Hello (%d)\n", ID);**: Fiecare thread va afișa "Hello" urmat de propriul său ID.
5. **printf("World (%d)\n", ID);**: Imediat după, fiecare thread va afișa "World" urmat de propriul său ID.

Proprietăți cheie ale ieșirii:

- **Număr de mesaje**: Vor fi exact 4 mesaje "Hello (ID)" și 4 mesaje "World (ID)", un total de 8 linii de ieșire.
- **Ordine internă**: Pentru fiecare thread, mesajul "Hello (ID)" va fi **întotdeauna** afișat înaintea mesajului "World (ID)" pentru același ID.
- **Ordine externă (inter-thread)**: Ordinea în care thread-urile își execută și își afișează perechile de mesaje ("Hello" și "World") este **nedeterminată**. Un thread poate termina ambele sale mesaje înainte ca un alt thread să înceapă să afișeze, sau mesajele pot fi intercalate.

Să verificăm opțiunile:

- **A.** Prezintă Hello (2) World (2) de două ori și lipsește complet Hello (3) World (3). Aceasta este **incorectă**.
- **B.** Prezintă Hello (0) urmat de World (1) și Hello (0) urmat de World (1). Aceasta înseamnă că ID-urile din perechile "Hello" și "World" nu corespund, ceea ce este imposibil deoarece fiecare thread își folosește propriul ID pentru ambele mesaje. Aceasta este **incorectă**.
- **C.** Prezintă doar 4 linii de ieșire (ar trebui să fie 8) și are o eroare de tipar (world în loc de World). De asemenea, ordinea (Hello (0) world (1)) este incorectă din cauza ID-urilor care nu se potrivesc. Aceasta este **incorectă**.
- **D. None of the existing answers.**
 - Vom verifica dacă E este corectă.
- **E.**
 - Hello (0)
 - World (0)
 - Hello (2)
 - World (2)
 - Hello (1)
 - World (1)
 - Hello (3)
 - World (3)
 - Toate cele 4 ID-uri de thread (0, 1, 2, 3) sunt prezente.
 - Pentru fiecare thread, "Hello (ID)" este afișat înaintea "World (ID)".
 - Ordinea perechilor de la thread-uri diferite (0, apoi 2, apoi 1, apoi 3) este o interleaving validă și posibilă într-un program paralel.
 - Această ieșire este o **ieșire potențială și validă**.

Răspunsul corect este **E**.